

Logging parte 2 & Aspect Oriented Programming

Logging

Como vimos en clase vimos que logback, antes de intentar cargar la configuración de `logback.xml` va a verificar si existe `logback-test.xml`. Esto es así para permitir que tengamos una configuración separada bajo `src/test/resources` y así en los unit tests poder loggear por separado, generalmente consola.

Nosotros sin embargo, buscamos como tener una configuración distinta entre cuando levantamos Jetty desde Eclipse para desarrollo y cuando generamos un war para deployar en Tomcat, por lo que la solución no nos sirve directamente, pero podemos aprovechar esto para generarnos algunas alternativas. Lo que vamos a hacer es tener 2 archivos de configuración, producción (`logback.xml`) y desarrollo (`logback-test.xml`) ambos en `src/main/resources`, de modo que en desarrollo siempre encuentre primero la configuración de desarrollo, pero luego configurar Maven para dejar afuera del war generado el archivo `logback-test.xml` así en producción siempre usa la configuración definida en `logback.xml`. Esta no es la única solución posible (existen otras usando por ejemplo profiles de Maven y filtering), pero de momento no vamos a profundizar en esto.

Lo primero que hacemos es renombrar `logback.xml` a `logback-test.xml` dentro de la misma carpeta `src/main/resources`, dejando

así el `ConsoleAppender` que usabamos la clase pasada para desarrollo.

Luego vamos a generar un nuevo `logback.xml` en `src/main/resources` con una configuración más acorde a producción, es decir, logueando a archivos, pero asegurandonos de que estos logs no crezcan infinitamente (dejando al servidor sin espacio en disco), y sobre todo, que no esté logueando en un nivel demasiado bajo.

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
    <property name="defaultPattern" value="%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n" />

    <appender name="root-appender" class="ch.qos.logback.core.rolling.RollingFileAppender">
        <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
            <maxHistory>5</maxHistory>
            <fileNamePattern>logs/warnings.%d{yyyy-MM-dd}.log</fileNamePattern>
        </rollingPolicy>
        <encoder>
            <pattern>${defaultPattern:-}</pattern>
        </encoder>
    </appender>
```

```

        <appender name="paw-appender" class="ch.qos.logback
ck.core.rolling.RollingFileAppender">
            <rollingPolicy class="ch.qos.logback.core
.rolling.TimeBasedRollingPolicy">
                <maxHistory>5</maxHistory>
                <fileNamePattern>logs/paw.%d{yyyy
-MM-dd}.log</fileNamePattern>
            </rollingPolicy>
            <encoder>
                <pattern>${defaultPattern:-}</pattern>
            </encoder>
        </appender>

        <root level="WARN">
            <appender-ref ref="root-appender" />
        </root>

        <logger name="ar.edu.itba" level="DEBUG" additivity="false">
            <appender-ref ref="paw-appender"/>
        </logger>
    </configuration>

```

Luego, en el pom padre, configuramos el plugin que genera el war para omitir la configuración de desarrollo

```

<plugin>

```

```
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-war-plugin</artifactId>
<version>${maven-war-plugin.version}</version>
<configuration>
  <packagingExcludes>*/logback-test.xml</packagingExcludes>
</configuration>
</plugin>
```

definiendo la versión del plugin como

```
<maven-war-plugin.version>2.6</maven-war-plugin.version>
```

Corremos desde Eclipse y vemos en consola los logs, armamos un war con `mvn package` y vemos que dentro del war generado no se incluye `logback-test.xml` en la carpeta `WEB-INF/classes/`, pero si `logback.xml` tal cual esperabamos.

Finalmente, estamos en condiciones de agregar nuestros propios logs en nuestra aplicación. Para esto, debemos en la clase donde queramos loguear cosas, definir un `Logger`, lo que en el caso del `UserController` se hace de la siguiente manera:

```
private static final Logger LOGGER = LoggerFactory.getLogger(UserController.class);
```

Tener cuidado de usar las clases `Logger` y `LoggerFactory` de SLF4J, así somos independientes de la implementación concreta que se esté utilizando.

Luego podríamos loguear mensajes en cualquier log level usando los métodos provistos por el logger. Recordar no concatenar `String`s ya que es muy ineficiente. En lugar de eso, podemos usar como placeholder `{}` y pasar cualquier objeto para reemplazarlos como argumentos variables. Por ejemplo podríamos loguear el usuario actualmente logueado en cada request de este controller modificando `loggedUser` de la siguiente manera.

```
@ModelAttribute
public User loggedUser() {
    final Authentication auth = SecurityContextHolder.getContext().getAuthentication();
    final User user = us.getByUsername((String) auth.getName());
    LOGGER.debug("Logged user is {}", user);
    return user;
}
```

Aspect Oriented Programming

Analicemos que ocurre si queremos agregar transacciones a nuestra aplicación. Vimos que podríamos hacerlo a mano ejecutando las queries correspondientes (begin, commit y rollback) usando `jdbcTemplate`, pero hacer esto sería engorroso, propenso a errores y

repetitivo. Analizando en detalle lo que queremos hacer, vimos que no es otra cosa que decorar las llamadas al DAO con la lógica necesaria para que esta llamada se haga dentro de una transacción, de la forma:

```
try {
    beginTransaction();
    callMyOwnMethod();
    doCommit();
} catch (final Throwable e) {
    doRollback();
    throw e;
}
```

Analizamos también, que esto lo queremos método por método, y que este comportamiento es cross a múltiples clases del sistema. Para no repetir código, lo natural sería usar herencia con una suerte de *AbstractDao* padre, pero esto tiene ciertos problemas. Por un lado, estamos haciendo que los DAOs estén relacionados entre sí, cuando no comparten nada salvo el no querer repetir código. Por otro lado, dado que sólo podemos extender una clase, tomar esta decisión pone serias restricciones a futuro a nuestra aplicación.

Este tipo de funcionalidades cross, que no se mapean a herencia (funciones como logs de auditoria, medir performance de métodos, transacciones, adquirir locks sobre recursos compartidos, etc.), sino que son *transversales* a la jerarquía de clases, son los que dispararon un paradigma de programación distinto llamado *Aspect Oriented*

Programming (AOP). Esta necesidad de “reusar código horizontalmente” ha dado origen a otros conceptos a lo largo de los años como **traits**, presentes en lenguajes como **Groovy**, **PHP**, e implementables via default methods de **Java 8**.

En Spring, AOP funciona por medio de annotations. Las clases y métodos que deseamos tengan un cierto comportamiento son anotadas y luego Spring se encarga de proveerlo en runtime. Para esto hay dos formas de implementarlo:

1. la default, es al crear una instancia de la clase anotada para inyectarla, crea un **Proxy** que decora la implementación real e inyecta el proxy. Esto es transparente para toda la aplicación (cumple con la misma interfaz pública), y permite interceptar toda llamada a método para agregar el comportamiento deseado.
2. usar *weaving*, es decir, reescribir el bytecode de la clase al momento de cargarse la clase a memoria. Requiere un classloader custom y es mucho más difícil de configurar, pero es más eficiente (no requiere reflection) y no tiene restricciones, como sí tiene el método de proxy, respecto de donde deben estar los annotations.

Nosotros vamos a usar los proxys por su simpleza, y por que de momento la performance no es un concern para nosotros. Dado que son proxies que decoran la implementación real cumpliendo la misma interfaz tenemos 2 restricciones a tener en cuenta:

1. el proxy sólo conoce los métodos públicos (interfaz), los annotations en métodos que no son parte de la interfaz son ignorados en silencio.
2. al ser un patrón decorator, sólo intercepta las llamadas que se hacen desde afuera. Si la implementación decorada llama a un método propio, a través de la referencia `this` que tiene, el mismo no pasa por el proxy y no tendrá el comportamiento adicional esperado.

Con estas dos consideraciones, estamos en condiciones de habilitar AOP en nuestro proyecto y configurar transacciones por medio del mismo.

Empezamos agregando la dependencia en el pom padre.

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-tx</artifactId>
  <version>${org.springframework.version}</version>
</dependency>
```

Y luego agregamos esta misma dependencia en el módulo de servicios, ya que desde allí vamos a querer controlar el scope de nuestras transacciones:

```
<dependency>
  <groupId>org.springframework</groupId>
```



```
<artifactId>spring<-tx/artifactId>  
</dependency>
```

Corremos `mvn eclipse:eclipse` y refrescamos y ya estamos en condiciones de empezar a agregar el annotation `@Transactional` a clases y métodos del servicio. Si anotamos la clase, todos los métodos públicos serán ejecutados dentro de una transacción (tener en cuenta que esto es independiente de si realmente están haciendo queries contra la base de datos en dichos métodos).

Ahora debemos habilitar AOP y configurar quien va a manejar las transacciones. Para esto editaremos `WebConfig` en el módulo webapp de la siguiente manera:

1. anotaremos la clase `WebConfig` con el annotation `@EnableTransactionManagement`
2. agregaremos un método para definir nuestro transaction manager

```
@Bean  
public PlatformTransactionManager transactionManager(  
final DataSource ds) {  
    return new DataSourceTransactionManager(ds);  
}
```

Con esto ya podemos anotar nuestras clases y métodos con

`@Transactional` para que corran dentro de transacciones.

La transaccionabilidad puede darse a nivel DAO o a nivel Service. La ventaja de hacerlo a nivel Service, es que nos permite revertir operaciones en la base de datos, si fallan operaciones de negocio que no necesariamente operan sobre la base de datos (por ejemplo, enviar un mail o llamar a un webservice).

Del mismo modo, usar transacciones via AOP nos permite mayor facilidad en el armado de tests. Por ejemplo, ahora podemos usar el annotation `@Rollback` en nuestros tests, para indicarle a Spring que los métodos deben correr dentro de transacciones, y que al finalizar, deben ser rollbackeadas (se evita así el tener que hacer un cleanup de los cambios que pueda producir mi test en forma manual).

Spring ofrece múltiples annotations para todos los casos comunes de aspectos, pueden investigarlos e incorporar los que precisen. En clase vimos en particular `@Scheduled` para permitir ejecutar métodos en forma automática en forma recurrente.